



DSTALGO BSIS01

Name: Mark Justine V. Dela Cruz	Score:
Section: BSIS01	Date: 2026/9/29

A. Data Structures and Algorithms

A data structure is a format for organizing, storing, and managing data in memory. An algorithm is a step-by-step procedure for solving a problem or executing a task.

Overview/Example: Storing envelopes alphabetically on a shelf is the data structure; following the steps to find a specific envelope is the algorithm.

B. Linear Data Structure (One-Dimension Array)

A linear data structure arranges data elements sequentially in a single line, where each element connects to its previous and next neighbor.

Overview/Example: A queue at a check-out counter where people are processed one after another in the order they arrived.

A one-dimension array is a fixed-size linear structure storing elements of the same data type in contiguous memory.

Overview/Example: A row of indexed containers, such as scores = [80, 90, 85]. Accessing index 1 gives 90 directly.

C. The Big O Notation

Big O Notation is a way to measure algorithm efficiency. tells us how much time or memory an algorithm needs as the input size (n) gets bigger, focusing mainly on the WORST CASE scenario

Overview/Example: Checking every single item in a list one by one scales linearly as $O(n)$. But if you keep cutting the search area in half each time, it scales way better as $O(\log n)$.



DSTALGO BSIS01

D. Basic Searching Algorithms (Linear Search and Binary Search)

Basic Searching Algorithms are just procedures used to find a specific target element inside a collection of data.

Linear search: goes through an array value by value from start to finish until it finds the target or runs out of items. It doesn't need the list to be sorted.

Overview/Example: We are Looking for 3 in [5, 12, 8, 3]. You check 5, then 12, then 8, and finally 3. If it's at the very end, you end up checking all n items, which gives it a worst-case time complexity of $O(n)$.

Binary search: finds a target in a sorted array by checking the middle element and cutting the remaining search area in half after every single step.

Overview/Example: So we are looking for 30 in [10, 20, 30, 40]. We check the middle element 20. Since 30 is bigger, you toss out the left half completely and check the remaining right side. This makes it super fast with a time complexity of $O(\log n)$.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press. <https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/>
2. GeeksforGeeks. (2023). *What is Data Structure?* <https://www.geeksforgeeks.org/dsa/data-structure-meaning/>
3. W3Schools. (2023). *Data Structures and Algorithms Tutorials*. <https://www.w3schools.com/dsa/>